The University of Greenwich

School of Computing and Mathematical Sciences

COMP1845 Systems Development Coursework



Student ID: 001474790

Word Count: 2114

Tutor: Matthew Prichard

Contents

Part 1 (Design and Implementation of Database System) 2

A: Identifying Entities, Attributes and Datatypes 2

B: Entity Relationship Diagram 4

C: Normalisation 5

D: Implementation in phpMyAdmin 6

E: Insertion of Records 7

Part 2 (Data Manipulation & Testing) 8

F: Student Contact Details 8

G: Student Countries 9

H: Surname Lookup 11

I: Review Student Bookings of Nationalities 11

J: Room Price Lookup 11

Part 1 (Design and Implementation of Database System)

A: Identifying Entities, Attributes and Datatypes

Students

This entity stores personal and contact details about each student living in the university accommodation. In database terms, a student is an entity meaning it represents a real-world object the system needs to store data about (Connolly & Begg, 2015) It allows staff to identify students, view their nationality and contact them if needed.

Attributes:

- student_id (Primary Key) - a unique number given to each student. A primary key is used to identify each record in a table (W3Schools, 2024a)
- first_name – The student's first or given name.
- last_ name – the students family name.
- dob – the student's date of birth.
- nationality – the student's nationality (used for reporting).
- email – the students university email address.
- phone – a contact phone number.
- address – the student's home or term-time address.

Room Types

This entity lists the different categories of rooms available, such as single rooms, ensuite rooms, or studios. Each attribute describes a characteristic of the room type (Connolly & Begg, 2015)

Attributes:

- roomtypes_id (Primary Key) - unique ID for each room type.
- type – name of the room type (e.g. Single, En-suite).
- weekly_rent – the rent charged per week for this type of room.
- features – a short description of what is included (e.g. private bathroom, kitchen).

Rooms

This entity holds details about each physical room in the accommodation building. It links each room to a specific room type and tracks whether the room is currently vacant or occupied. Linking two tables using a shared field creates a relationship using a foreign key (W3Schools, 2024b)

Attributes:

- Room_id (Primary Key) - unique number for each room.
- Room_number – the label or door number of the room.
- Roomtype_id (Foreign Key) - links to the Room Types table.
- Weekly_rent – rent changed for this specific room
- Occupancy_status – shows whether the room is vacant or occupied.

Facilities:

This entity stores a list of facilities that can be provided within rooms. Examples include Wi-fi, heating, a wardrobe, or laundry access.

Attributes:

- Facility_id (Primary Key) - this is a unique number for each facility.
- Name – name of the facility: Wi-Fi, Laundry, Heating.

Room Facilities

This entity connects rooms and the facilities they contain. Because one room can have many facilities and each facility can appear in many rooms, this table represents a many to many relationship between rooms and facilities.

Attributes:

- Room_id (Foreign Key) - identifies the room.
- Facility_id (Foreign key) - identifies the facility

Bookings

This entity records which student has booked which room and for what dates. It also includes the rent and booking status so staff can manage contracts.

Attributes:

- Booking_id (Primary key) - unique booking number.
- Student_id (Foreign Key) - links to the student who made the booking.
- Room_id (Foreign Key) - links to the room being booked.
- Start_date – date the booking starts.
- End_date – date the booking ends.
- Weekley_rent – rent charged per week for the room
- Status - shows whether the booking is Active, Completed, or Cancelled.

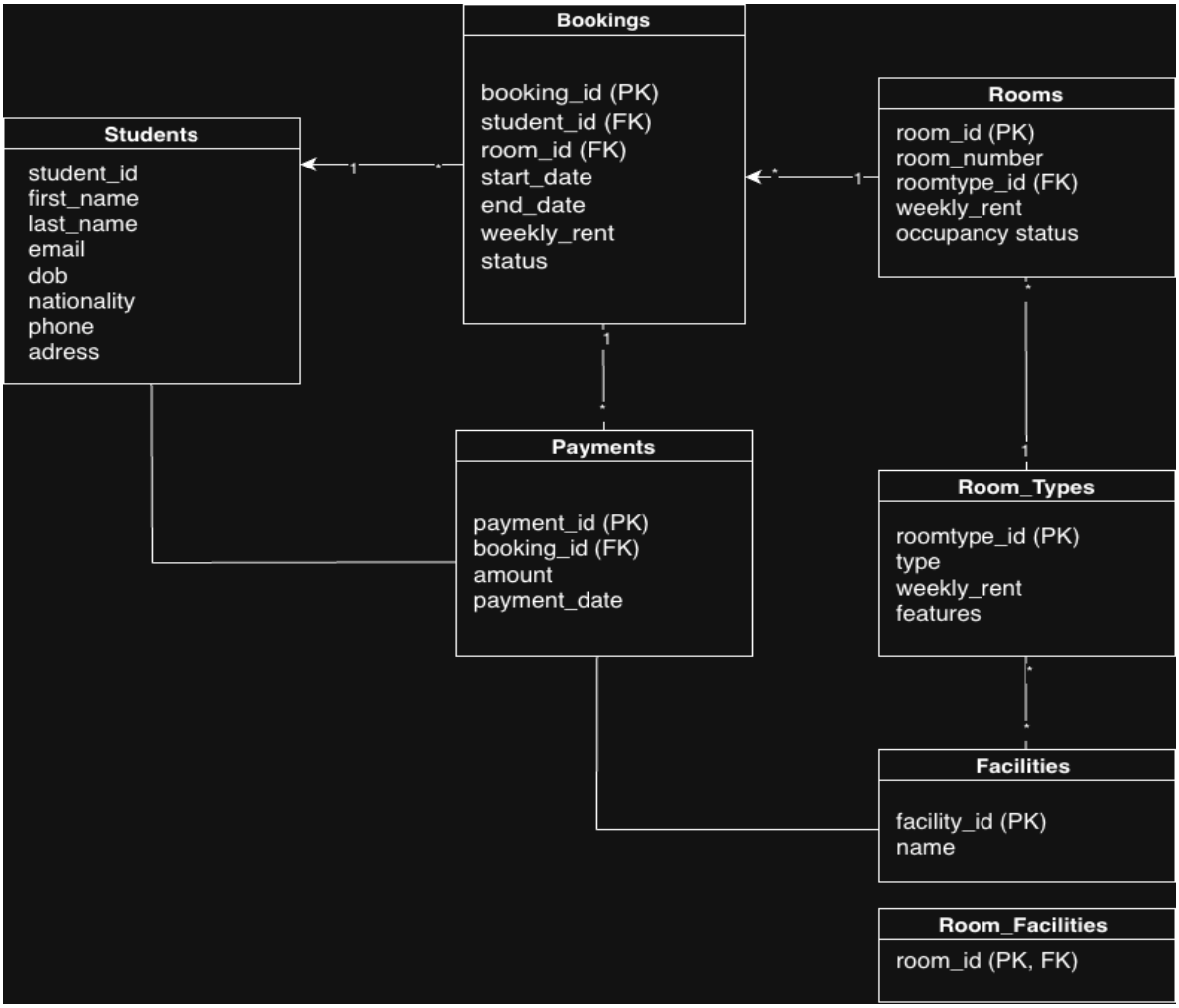
Payments

This entity keeps track of all rent payments made by students. It records how much was paid, when it was paid and how the payment was made.

Attributes:

- Payment_id (Primary Key) - unique number for each payment
- Booking_id (Foreign Key) - links the payment to the correct booking.
- Amount – the amount of money paid.
- Payment_date – the date the payment was made.
- Method – how the payment was made (e.g card, cash, bank transfer).

B: Entity Relationship Diagram



The ER diagram above shows the seven main parts of the student accommodation system: Students, Rooms, Room Types, Facilities, Room Facilities, Bookings and Payments. Each table (also called and entity) stores one type of information. Each table has a primary key, which is a unique ID number that identifies each record. (W3Schools, 2024a) Some tables also include Foreign Keys, which are ID numbers taken from another table so that the data is connected correctly. (W3Schools, 2024b)

For example, one student can make many Bookings, and each booking can have several payments. This is an example of a one-to-many relationship in database (Connolly & Begg, 2015). Each room belongs to one room type such as en-suite or standard. Rooms can also have several facilities (like wi-fi or heating) and because a facility can belong to many rooms too, they are linked through a middle table called Room facilities.
These links between the tables show how the real-life accommodation system works and helps keep the databases neat, accurate, and easy to update.

C: Normalisation

Normalisation is a process used to tidy up data in a database so that information is clear, well organised, and not repeated. (datacamp, 2024) It helps make the database easier to manage and reduces the chance of mistakes when updating data. The student accommodation database has been designed up to Third Normal Form (3NF) to make sure each table only stores on type of information.

First Normal Form (1NF)
In 1NF, each field should hold only one value. A table cannot contain repeating groups or multiple values inside a single cell. (W3Schools) For example, instead of storing multiple facilities together like this:

Room	Facilities
101	WiFi, Heating

It is changed to:

Room	Facility
101	WiFi
101	Heating

This makes sure that each field contains a single piece of information.

Second Normal Form (2NF)

In 2NF, data is placed in the correct table so that all non-key attributes depend on the whole primary key, not just part of it (Connolly & Begg, 2015) for example, details about room type and price are moved into a separate Room_Types table:

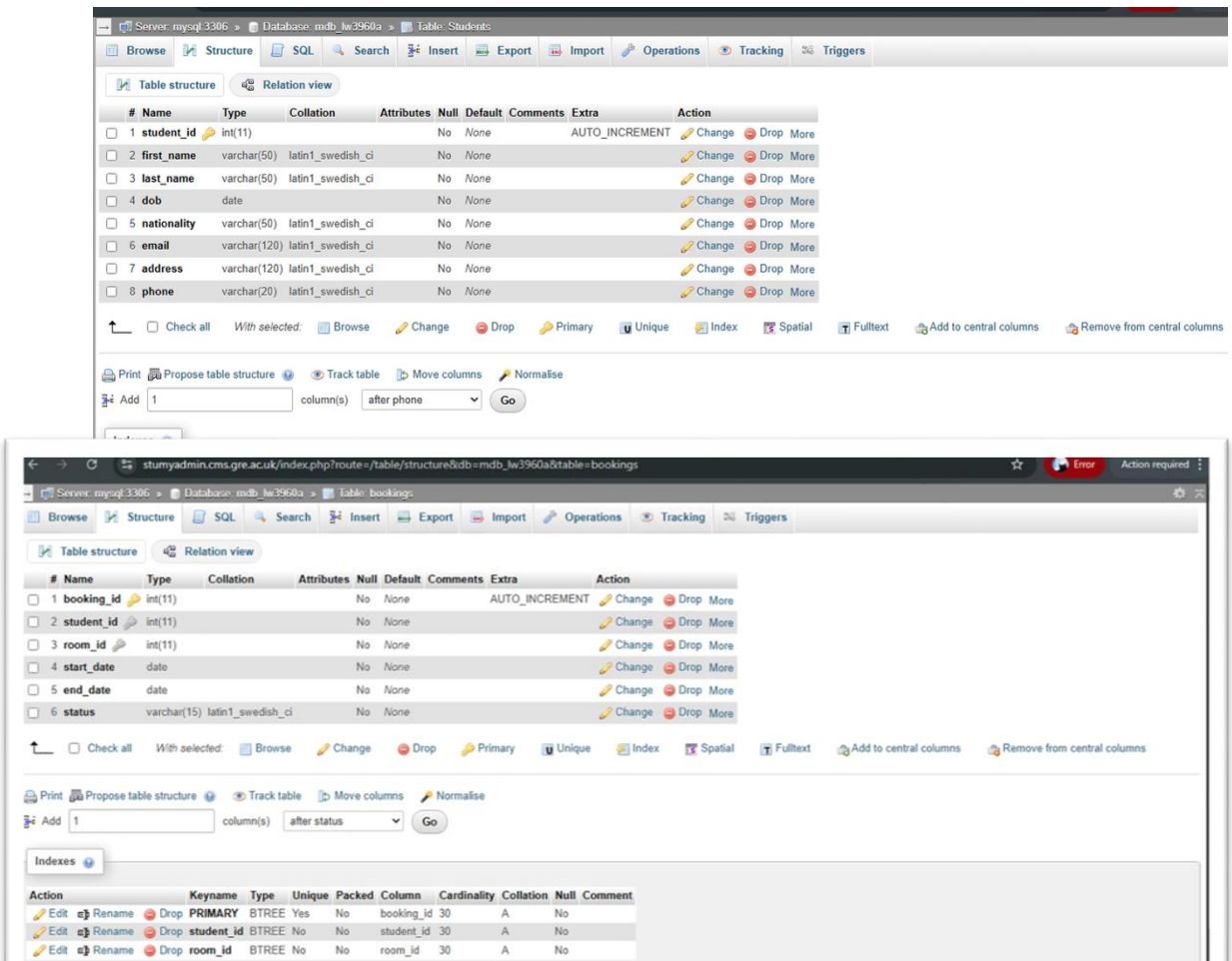
Roomtype_id	Type_name	Price
1	En-suite	£500
2	Standard	£400

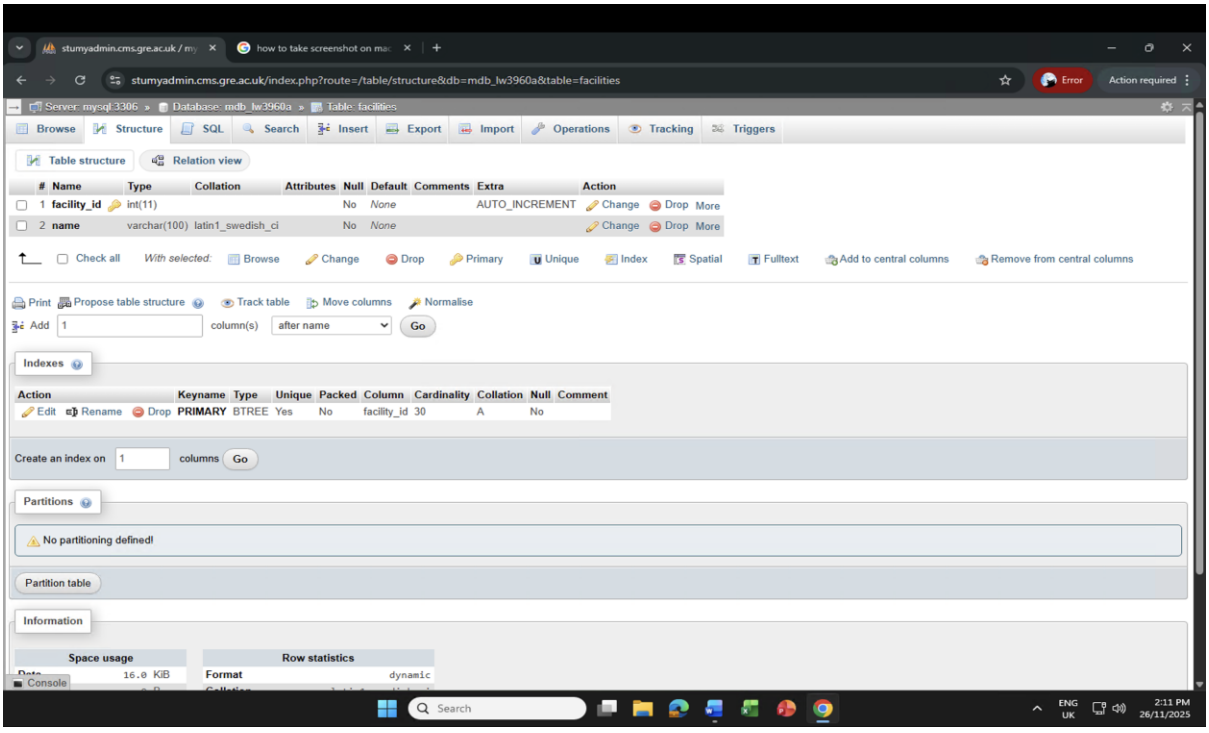
Third Normal Form (3NF) each table contains data about one topic only, and no attribute should depend on anything other than the primary key (W3Schools,2024b) Facilities are stored in the Facilities table and linked to rooms through Room_Facilites, so the design stays simple and consistent.

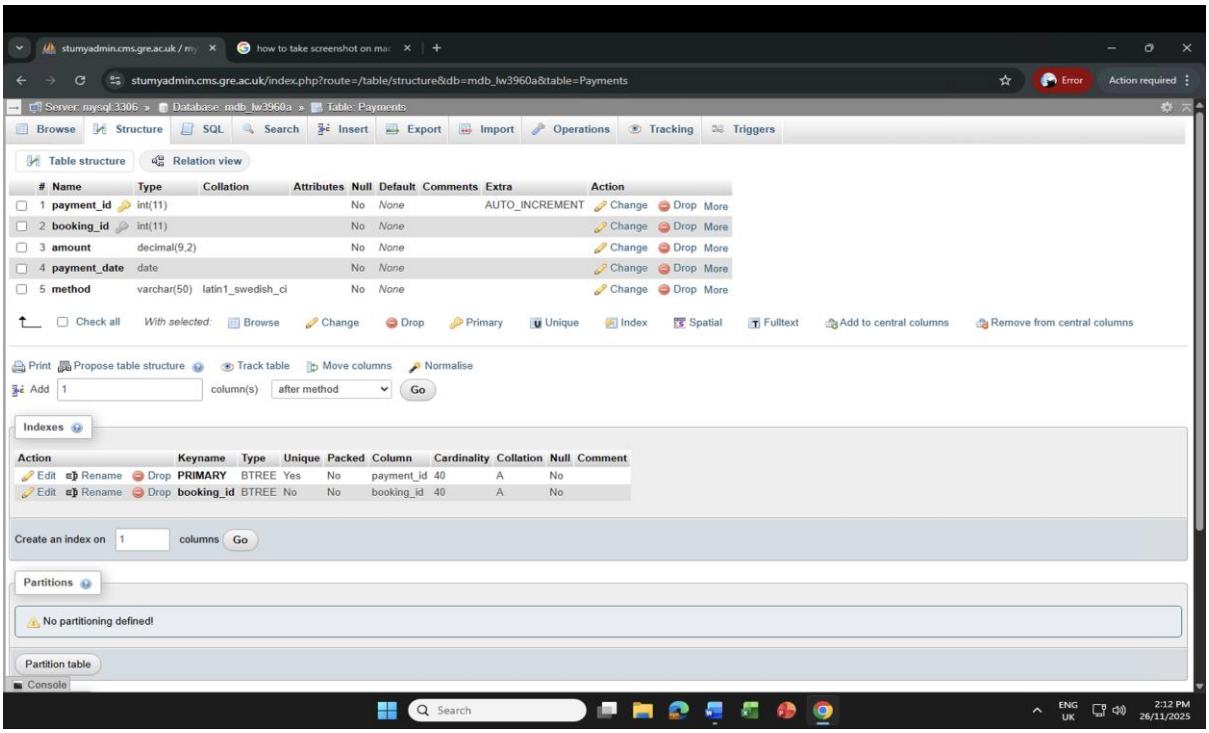
Without normalisation, the same details (like prices or facilities) would have to be entered many times, which can cause errors. By following these three stages the system avoids duplication, keeps data accurate and makes updates faster and easier creating a professional, efficient database design.

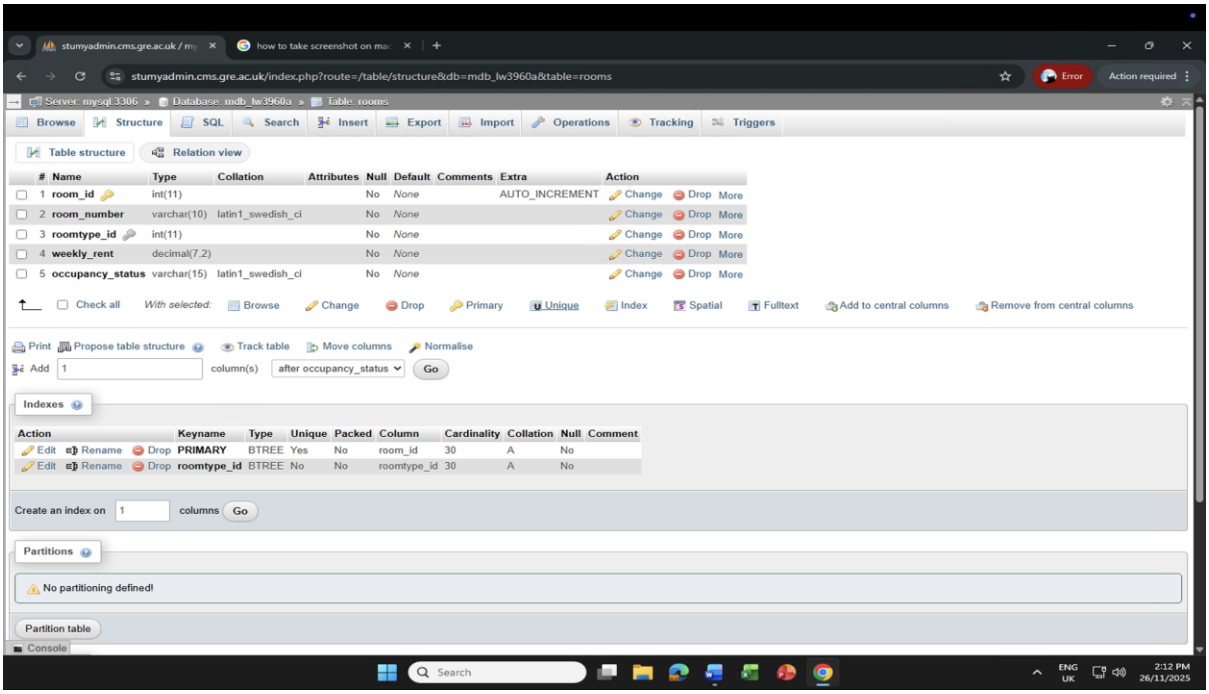
D: Implementation in phpMyAdmin

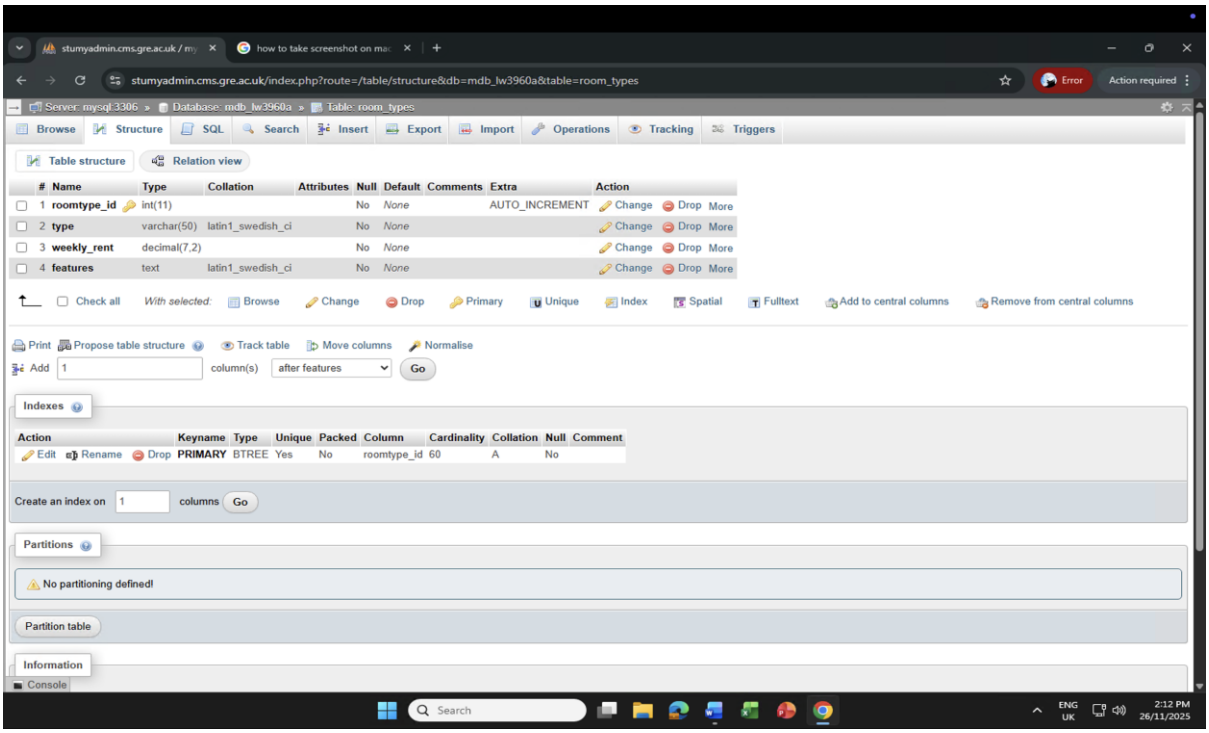
For this part of the coursework, I started building my database inside phpMyAdmin based on the design from the earlier questions. The aim is to create all the tables and then up the relationships between them by using things like primary keys and foreign keys. Referential integrity means, making sure that the links between tables are valid, such as not allowing a booking to be added if the student doesn't exist (Connolly and Begg, 2015). Foreign keys help control this by forcing the database to match records correctly, which stops the system from ending up with missing or incorrect data (Rob and Coronel,2019). As explained in MySQL documentation, foreign keys are essential for controlling how records in related tables interact, especially when inserting, updating or deleting data. (MySQL, 2024)




stumyadmin.cms.gre.ac.uk / my
how to take screenshot on mac
stumyadmin.cms.gre.ac.uk/index.php?route=/table/structure&db=mdb_lw3960a&table=facilities
Error
Action required
Server: mysql:3306 » Database: mdb_lw3960a » Table: facilities
Browse
Structure
SQL
Search
Insert
Export
Import
Operations
Tracking
Triggers
Table structure
Relation view
Name Type Collation Attributes Null Default Comments Extra Action
1 facility_id int(11) No None AUTO_INCREMENT Change Drop More
2 name varchar(100) latin1_swedish_ci No None Change Drop More
Check all
With selected:
Browse
Change
Drop
Primary
Unique
Index
Spatial
Fulltext
Add to central columns
Remove from central columns
Print
Propose table structure
Track table
Move columns
Normalise
Add
1
column(s)
after name
Go
Indexes
Action Keyname Type Unique Packed Column Cardinality Collation Null Comment
Edit Rename Drop PRIMARY BTREE Yes No facility_id 30 A No
Create an index on
1
columns
Go
Partitions
No partitioning defined!
Partition table
Information
Space usage
16.0 KiB
Row statistics
Format
dynamic
Console
Search
ENG
UK
2:11 PM
26/11/2025


stumyadmin.cms.gre.ac.uk/index.php?route=/table/structure&db=mdb_lw3960a&table=Payments
Server: mysql:3306 » Database: mdb_lw3960a » Table: Payments
Browse Structure SQL Search Insert Export Import Operations Tracking Triggers
Table structure Relation view
Name Type Collation Attributes Null Default Comments Extra Action
1 payment_id int(11) No None AUTO_INCREMENT Change Drop More
2 booking_id int(11) No None Change Drop More
3 amount decimal(9,2) No None Change Drop More
4 payment_date date No None Change Drop More
5 method varchar(50) latin1_swedish_ci No None Change Drop More
Check all With selected: Browse Change Drop Primary Unique Index Spatial Fulltext Add to central columns Remove from central columns
Print Propose table structure Track table Move columns Normalise
Add 1 column(s) after method Go
Indexes
Action Keyname Type Unique Packed Column Cardinality Collation Null Comment
Edit Rename Drop PRIMARY BTREE Yes No payment_id 40 A No
Edit Rename Drop booking_id BTREE No No booking_id 40 A No
Create an index on 1 columns Go
Partitions
No partitioning defined!
Partition table
Console


stumyadmin.cms.gre.ac.uk/index.php?route=/table/structure&db=mdb_lw3960a&table=rooms
Server: mysql:3306 » Database: mdb_lw3960a » Table: rooms
Browse Structure SQL Search Insert Export Import Operations Tracking Triggers
Table structure Relation view
Name Type Collation Attributes Null Default Comments Extra Action
1 room_id int(11) No None AUTO_INCREMENT Change Drop More
2 room_number varchar(10) latin1_swedish_ci No None Change Drop More
3 roomtype_id int(11) No None Change Drop More
4 weekly_rent decimal(7,2) No None Change Drop More
5 occupancy_status varchar(15) latin1_swedish_ci No None Change Drop More
Check all With selected: Browse Change Drop Primary Unique Index Spatial Fulltext Add to central columns Remove from central columns
Print Propose table structure Track table Move columns Normalise
Add 1 column(s) after occupancy_status Go
Indexes
Action Keyname Type Unique Packed Column Cardinality Collation Null Comment
Edit Rename Drop PRIMARY BTREE Yes No room_id 30 A No
Edit Rename Drop roomtype_id BTREE No No roomtype_id 30 A No
Create an index on 1 columns Go
Partitions
No partitioning defined!
Partition table
Console



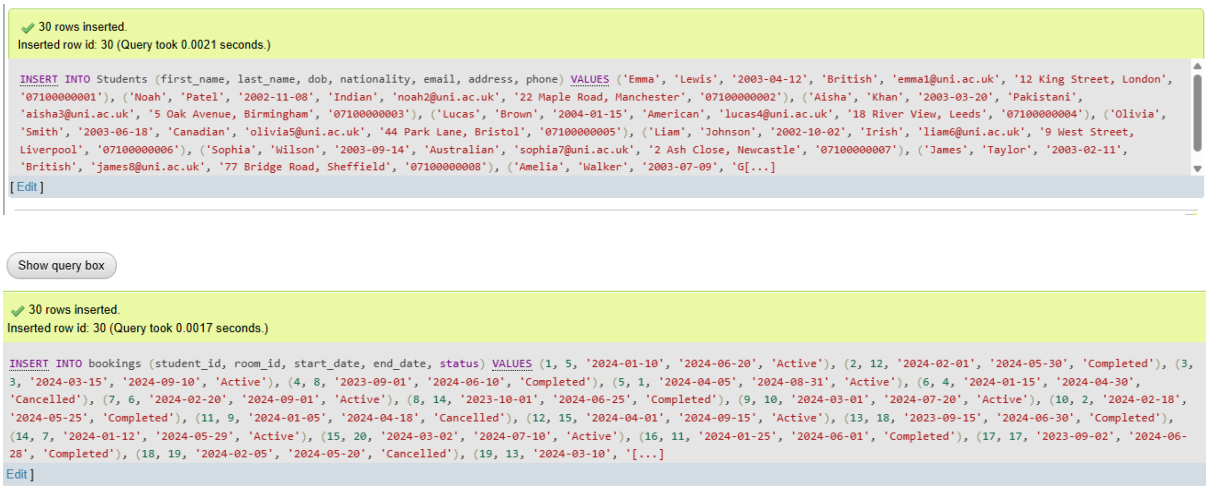
In phpMyAdmin, I created the required tables such as students, rooms, room_types, bookings, payments, facilities and room_facilities. For each table, I added a primary key (for example student_id in students or room_id in rooms) After that, I began adding the foreign key links using the relation view tool, which phpMyAdmin provides for creating these relationships visually (phpMyAdmin, 2024). I followed online resources such as W3Schools(2024) to help me understand where each foreign key should go and how they connect the different tables.

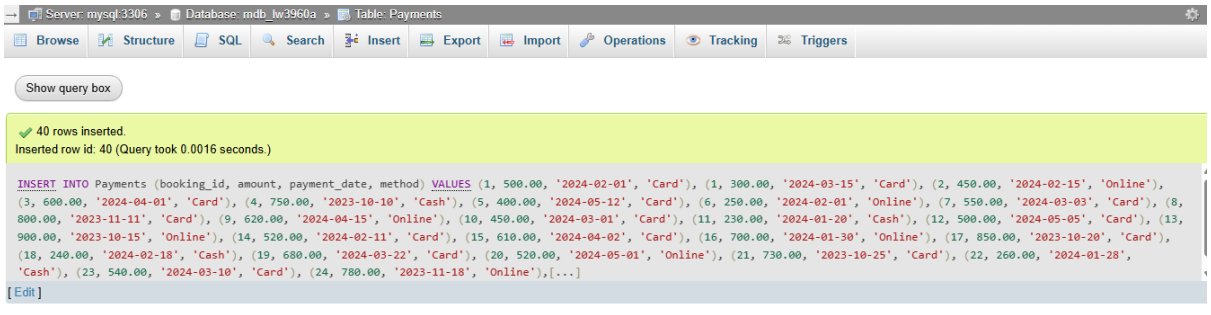
E: Insertion of Records

For this part of the coursework, I had to insert more than thirty records into every table in my database, and I did this using phpMyAdmin. I mainly used the SQL tab because once I understood how the INSERT INTO command worked, it became much quicker to add large amounts of data at once. I inserted realistic information into each table that matched the purpose of the system. For example, in the students table, I added different nationalities, emails, phone numbers and home addresses so it looked like a real university accommodation

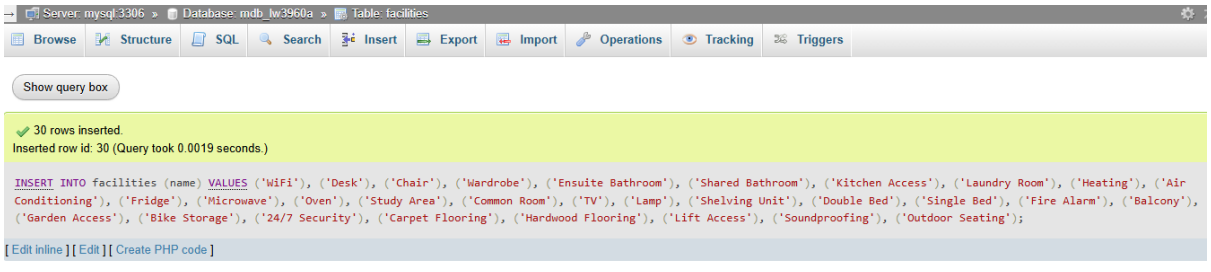
database. In the rooms and room_ types table I added different room numbers, types and rents to show the range of the accommodation a student can book.

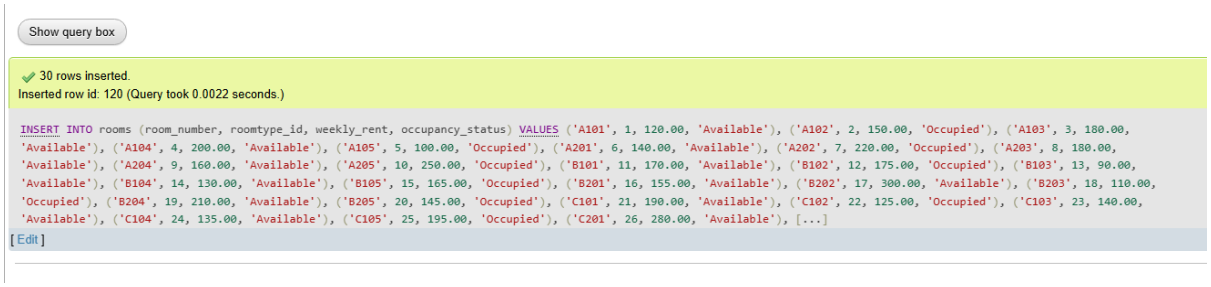
I also added linking data into the room_facilities table, which shows which facilities belong to which rooms. This is important because it helps the system answer real queries later like, “which rooms have Wi-Fi?” or “what facilities does RoomA101 have?”. The bookings table uses actual student IDs and room IDs, so it reflects real-life booking activity, and the payments table shows realistic payment dates and amounts. Each time I inserted the data, I took screenshots of the SQL I used and the results in the browse tab to show clear evidence of what I did, just like the task asked for. Overall, the inserted data helps the system behave like a proper accommodation database, because it supports searching, filtering and checking availability or payments in future queries (Connolly & Begg, 2015).

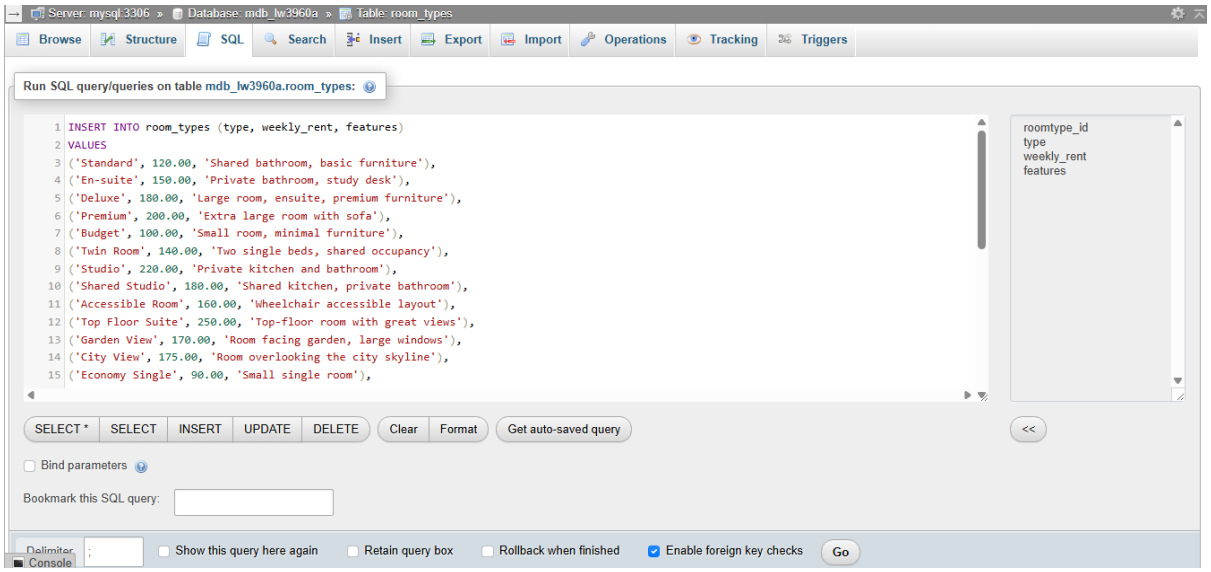



Server: mysql:3306 » Database: mdb_lw3960a » Table: Payments
Browse
Structure
SQL
Search
Insert
Export
Import
Operations
Tracking
Triggers
Show query box
40 rows inserted.
Inserted row id: 40 (Query took 0.0016 seconds.)
INSERT INTO Payments (booking_id, amount, payment_date, method) VALUES (1, 500.00, '2024-02-01', 'Card'), (1, 300.00, '2024-03-15', 'Card'), (2, 450.00, '2024-02-15', 'Online'), (3, 600.00, '2024-04-01', 'Card'), (4, 750.00, '2023-10-10', 'Cash'), (5, 400.00, '2024-05-12', 'Card'), (6, 250.00, '2024-02-01', 'Online'), (7, 550.00, '2024-03-03', 'Card'), (8, 800.00, '2023-11-11', 'Card'), (9, 620.00, '2024-04-15', 'Online'), (10, 450.00, '2024-03-01', 'Card'), (11, 230.00, '2024-01-20', 'Cash'), (12, 500.00, '2024-05-05', 'Card'), (13, 900.00, '2023-10-15', 'Online'), (14, 520.00, '2024-02-11', 'Card'), (15, 610.00, '2024-04-02', 'Card'), (16, 700.00, '2024-01-30', 'Online'), (17, 850.00, '2023-10-20', 'Card'), (18, 240.00, '2024-02-18', 'Cash'), (19, 680.00, '2024-03-22', 'Card'), (20, 520.00, '2024-05-01', 'Online'), (21, 730.00, '2023-10-25', 'Card'), (22, 260.00, '2024-01-28', 'Cash'), (23, 540.00, '2024-03-10', 'Card'), (24, 780.00, '2023-11-18', 'Online'),[...]
[Edit]

Console


Server: mysql:3306 » Database: mdb_lw3960a » Table: facilities
Browse
Structure
SQL
Search
Insert
Export
Import
Operations
Tracking
Triggers
Show query box
30 rows inserted.
Inserted row id: 30 (Query took 0.0019 seconds.)
INSERT INTO facilities (name) VALUES ('WiFi'), ('Desk'), ('Chair'), ('Wardrobe'), ('Ensuite Bathroom'), ('Shared Bathroom'), ('Kitchen Access'), ('Laundry Room'), ('Heating'), ('Air Conditioning'), ('Fridge'), ('Microwave'), ('Oven'), ('Study Area'), ('Common Room'), ('TV'), ('Lamp'), ('Shelving Unit'), ('Double Bed'), ('Single Bed'), ('Fire Alarm'), ('Balcony'), ('Garden Access'), ('Bike Storage'), ('24/7 Security'), ('Carpet Flooring'), ('Hardwood Flooring'), ('Lift Access'), ('Soundproofing'), ('Outdoor Seating');
[Edit inline] [Edit] [Create PHP code]

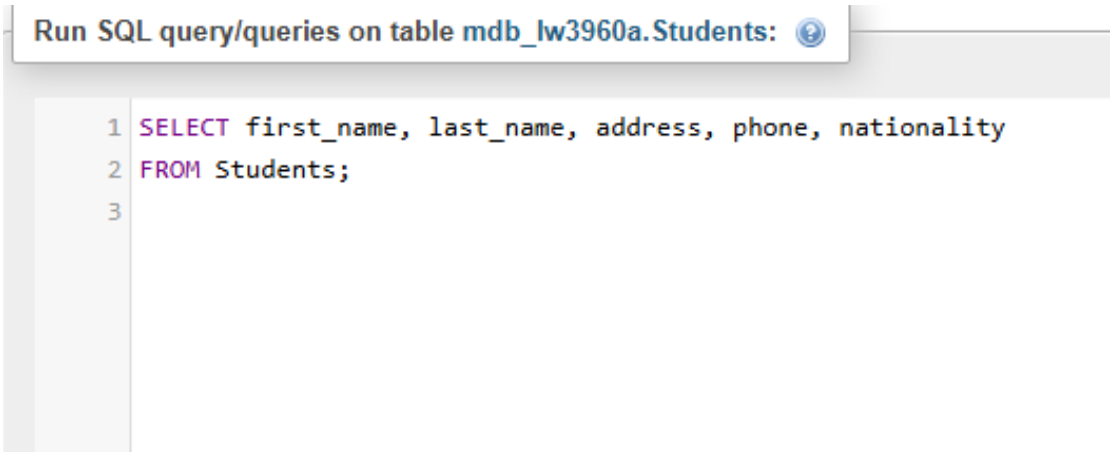

Show query box
30 rows inserted.
Inserted row id: 120 (Query took 0.0022 seconds.)
INSERT INTO rooms (room_number, roomtype_id, weekly_rent, occupancy_status) VALUES ('A101', 1, 120.00, 'Available'), ('A102', 2, 150.00, 'Occupied'), ('A103', 3, 180.00, 'Available'), ('A104', 4, 200.00, 'Available'), ('A105', 5, 100.00, 'Occupied'), ('A201', 6, 140.00, 'Available'), ('A202', 7, 220.00, 'Occupied'), ('A203', 8, 180.00, 'Available'), ('A204', 9, 160.00, 'Available'), ('A205', 10, 250.00, 'Occupied'), ('B101', 11, 170.00, 'Available'), ('B102', 12, 175.00, 'Occupied'), ('B103', 13, 90.00, 'Available'), ('B104', 14, 130.00, 'Available'), ('B105', 15, 165.00, 'Occupied'), ('B201', 16, 155.00, 'Available'), ('B202', 17, 300.00, 'Available'), ('B203', 18, 110.00, 'Occupied'), ('B204', 19, 210.00, 'Available'), ('B205', 20, 145.00, 'Occupied'), ('C101', 21, 190.00, 'Available'), ('C102', 22, 125.00, 'Occupied'), ('C103', 23, 140.00, 'Available'), ('C104', 24, 135.00, 'Available'), ('C105', 25, 195.00, 'Occupied'), ('C201', 26, 280.00, 'Available'), [...]
[Edit]

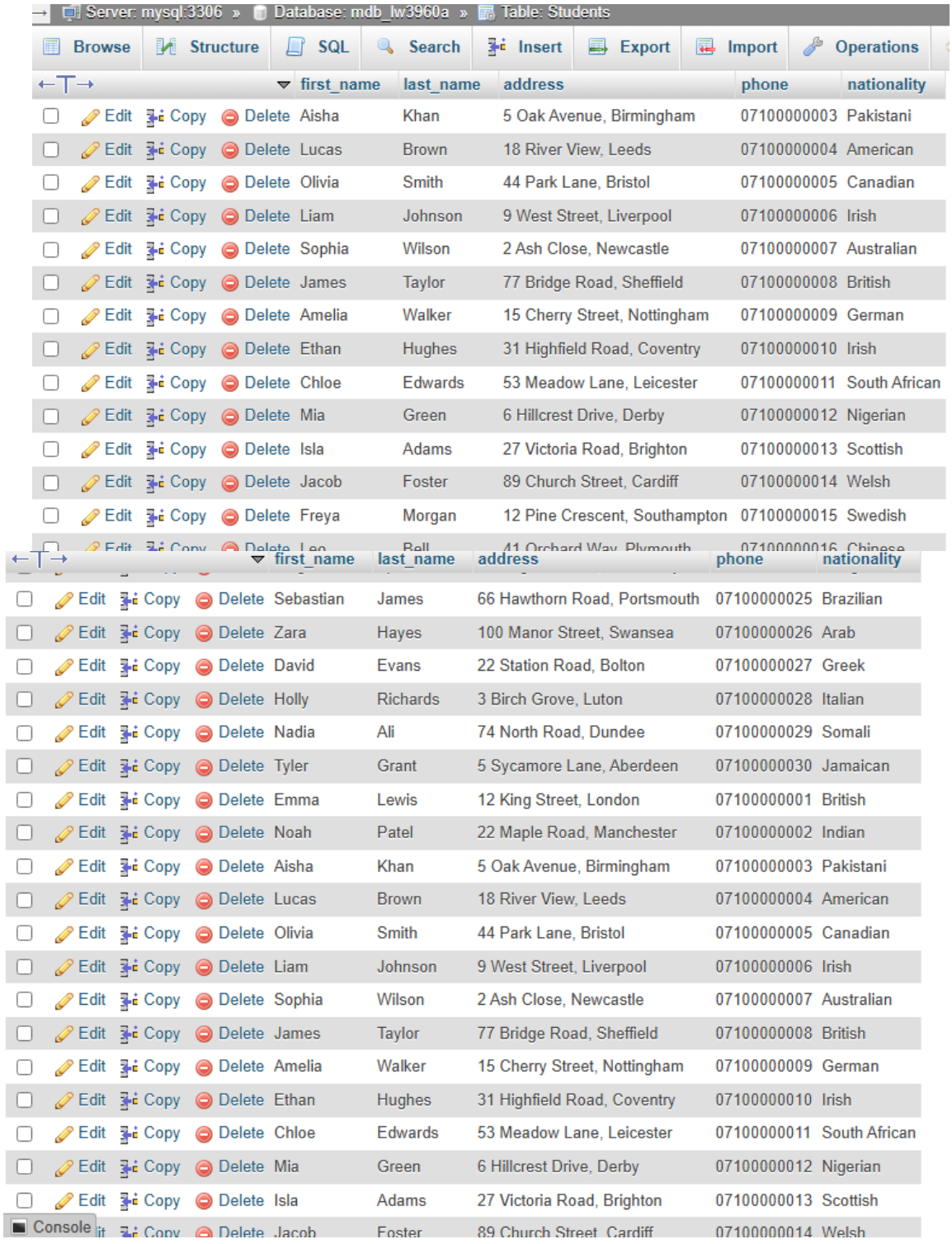


Part 2 (Data Manipulation & Testing)

F: Student Contact Details

Here is what I inserted into the SQL tab:



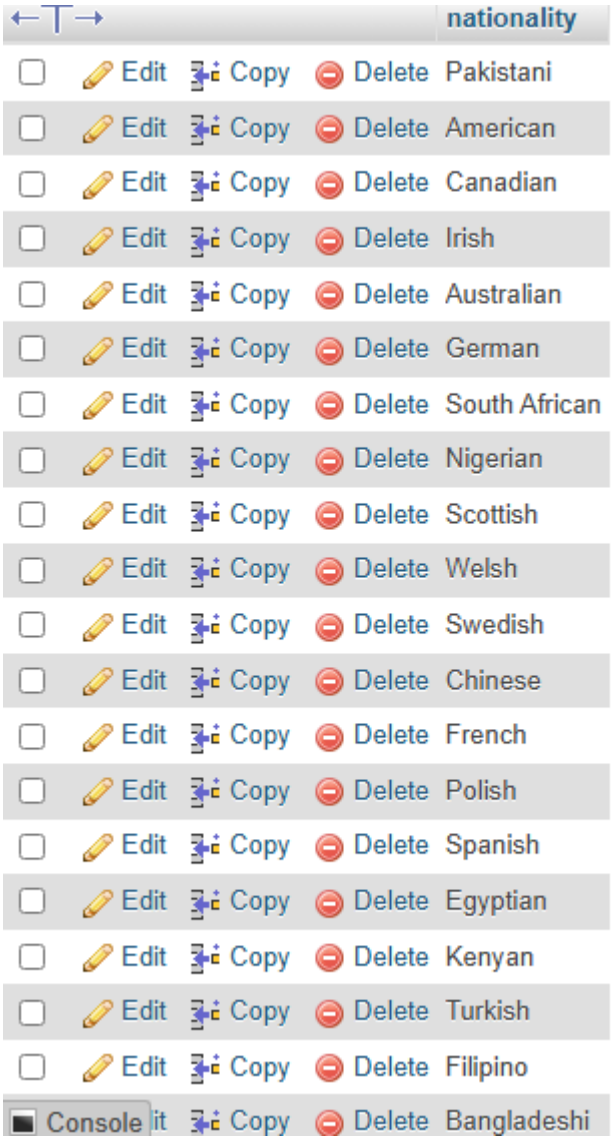


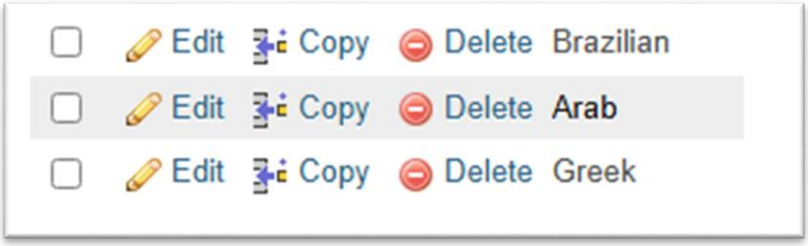
This query shows a full list of all the students who are registered in the system. I selected the main fields the office needs when they get calls from people asking for contact information. The SELECT part lists the exact columns I want to see (first name, last name, address, phone and nationality), and the FROM Students; tells my SQL to get

this information from the students table. This returns a clear list of every student with their contact details.

G: Student Countries

For this question, I had to show a list of all the different nationalities that are represented by the students currently in the system. Instead of showing every single student record, I only needed the unique nationalities. To do this, I used the SELECT DISTINCT keyword because it removes duplicate values and only returns one copy of each nationality, even if lots of students share the same one. This makes it easier for the accommodation office to quickly see which countries are represented without scrolling through the full student list. Using DISTINCT is exactly how SQL handles the situations where you only want unique data from a column (W3Schools,2024; Connolly and Begg,2015)





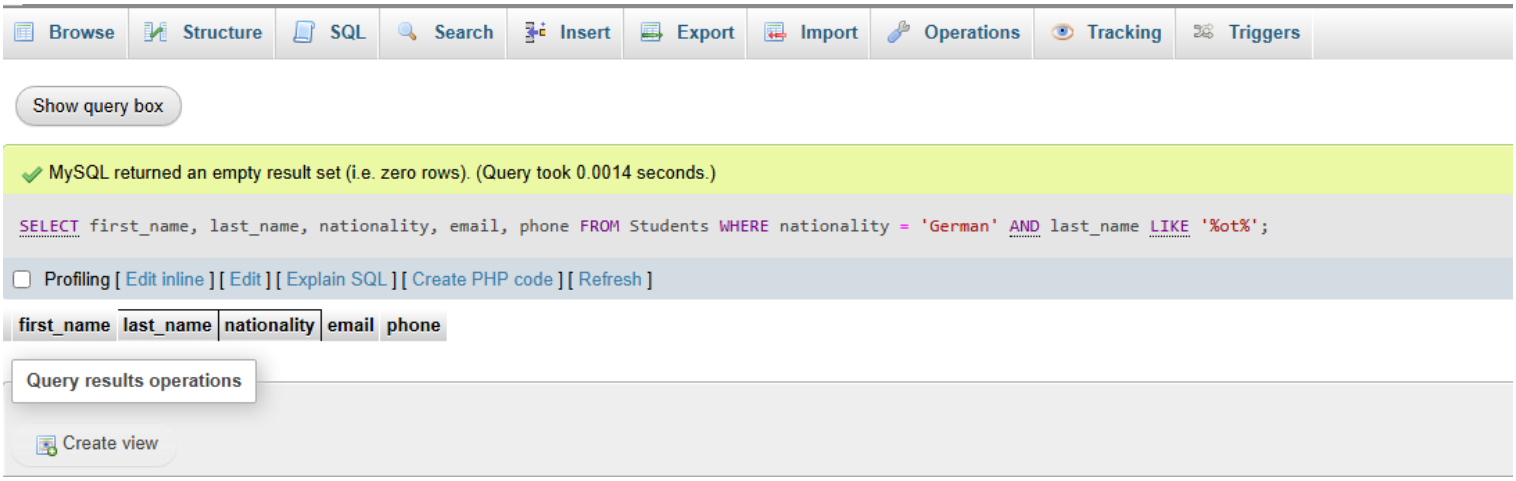

```
1 SELECT DISTINCT nationality
2 FROM Students;
3
```

H: Surname Lookup

In this question, I needed to help staff find a German student whose surname includes the letters “ot”, because they have fallen behind on payments. Since the exact surname is unknown, I used the LIKE operator with wildcards (%) on both sides of “ot”. This tells MySQL to look for any last name that contains “ot” anywhere inside it. I also added a condition to check that the student’s nationality is German. This way the query only returns German students whose surnames match the pattern (W3Schools, 2024)

Run SQL query/queries on table mdb_lw3960a.Students:

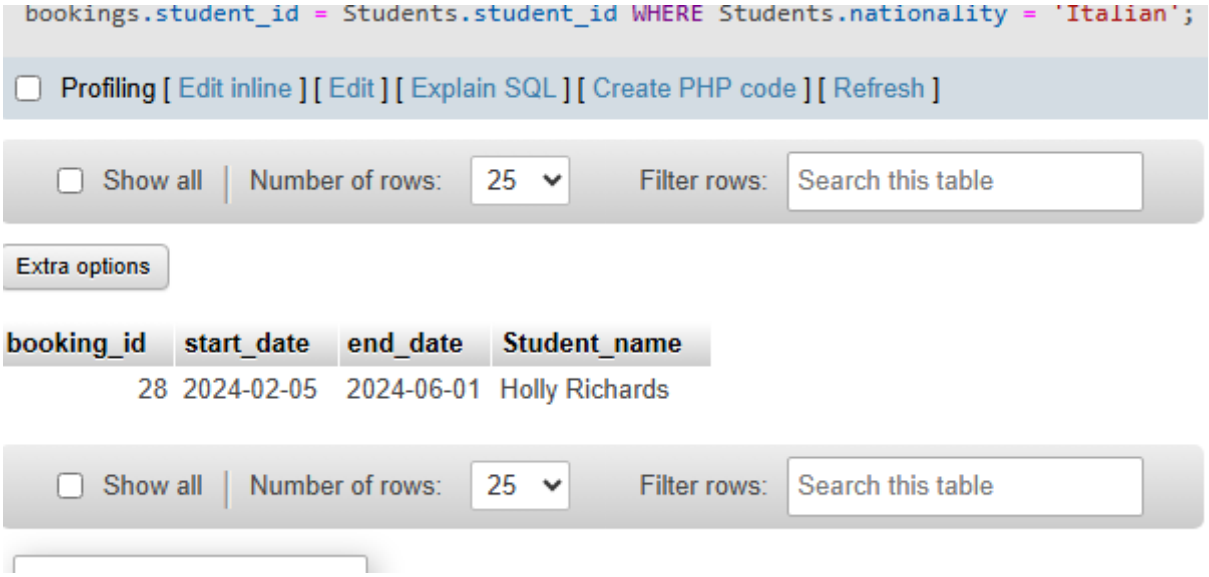
```
1 SELECT first_name, last_name, nationality, email, phone
2 FROM Students
3 WHERE nationality = 'German'
4 AND last_name LIKE '%ot%';
```



I: Review Student Bookings of Nationalities

Run SQL query/queries on table mdb_lw3960a.Students:

```sql
SELECT
bookings.booking_id,
bookings.start_date,
bookings.end_date,
CONCAT(Students.first_name, ' ', Students.last_name) AS Student_name
FROM bookings
INNER JOIN Students
    ON bookings.student_id = Students.student_id
WHERE Students.nationality = 'Italian';
```



For this task, I needed to find all bookings made by Italian students so the accommodation office can check which rooms they stayed in and during what dates. To do this, I used an INNER_JOIN to link the bookings table with the

students table, using the student_id that both tables share. The WHERE clause filters the results, so it only shows students whose nationality is Italian. I also used CONCAT to combine the students first and last name into one full name. The final result shows the booking ID, start date, end date and the student's name, which makes it easier for staff to match the complaint to the correct booking record (w3Schools, 2024; Connolly and Begg, 2015).

J: Room Price Lookup

For this part, I had to show all the rooms that fall within a certain rent range, which in my case was between £100 and £150. I joined the rooms table with the room_types table because the room type name is stored separately. The 'where' clause filters the results to only show rooms whose weekly rent is in that band. I also added a calculation to the SELECT list to work out what the rent would be with a 10% discount, by multiplying the weekly rent by 0.9. This lets the accommodation office quickly see which rooms can be promoted and what their discounted prices would be (W3Schools,2024)

```sql
SELECT
    rooms.room_id,
    room_types.type AS room_type,
    rooms.weekly_rent,
    rooms.weekly_rent * 0.9 AS discounted_rent
FROM rooms
INNER JOIN room_types
    ON rooms.roomtype_id = room_types.roomtype_id
WHERE rooms.weekly_rent BETWEEN 100 AND 150;

```

Extra options

room_id	room_type	weekly_rent	discounted_rent
91	Standard	120.00	108.000
92	En-suite	150.00	135.000
95	Budget	100.00	90.000
96	Twin Room	140.00	126.000
104	Economy Double	130.00	117.000
108	Attic Room	110.00	99.000
110	Compact Ensuite	145.00	130.500
112	Quiet Room	125.00	112.500
113	Courtyard Room	140.00	126.000
114	Shared Ensuite	135.00	121.500
117	Ground Floor Room	130.00	117.000
118	Basement Room	100.00	90.000
120	Minimalist Room	115.00	103.500

Reference list

Connolly, T.M. and Begg, C. (2015). *Database systems : a practical approach to design, implementation and management*. Harlow: Pearson Education Limited.

GeeksforGeeks (2025). *Introduction of ER Model*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/dbms/introduction-of-er-model/ [Accessed 22 Nov. 2025].

Gomstyn, A. and Jonker, A. (2025). *Database normalization*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/database-normalization [Accessed 23 Nov. 2025].

phpMyAdmin contributors (2025). *Documentation*. [online] phpMyAdmin. Available at: https://www.phpmyadmin.net/docs [Accessed 5 Nov. 2025].

Vinayak Baranwal (2025). *Database Normalization: 1NF, 2NF, 3NF & BCNF Examples*. [online] Digitalocean.com. Available at: https://www.digitalocean.com/community/tutorials/database-normalization [Accessed 5 Nov. 2025].

W3Schools (2019a). *SQL FOREIGN KEY Constraint*. [online] W3schools.com. Available at: https://www.w3schools.com/sql/sql_foreignkey.asp [Accessed 10 Nov. 2025].

W3Schools (2019b). *SQL LIKE Operator*. [online] W3schools.com. Available at: https://www.w3schools.com/sql/sql_like.asp [Accessed 17 Nov. 2025].

W3Schools (2019c). *SQL Normalization*. [online] W3schools.com. Available at: https://www.w3schools.com/sql/sql_foreignkey.asp [Accessed 10 Nov. 2025].

W3Schools (2024a). *SQL PRIMARY KEY Constraint*. [online] W3Schools. Available at: https://www.w3schools.com/sql/sql_primarykey.ASP [Accessed 5 Nov. 2025].

W3Schools (2024b). *SQL SELECT DISTINCT Statement*. [online] www.w3schools.com. Available at: https://www.w3schools.com/sql/sql_distinct.asp [Accessed 10 Nov. 2025].

W3Schools (2025). *SQL joins*. [online] W3schools.com. Available at: https://www.w3schools.com/sql/sql_join.asp [Accessed 17 Nov. 2025].

w3schools (2019). *SQL INSERT INTO Statement*. [online] W3schools.com. Available at: https://www.w3schools.com/sql/sql_insert.asp [Accessed 10 Nov. 2025].

W3schools.com. (2019). *SQL BETWEEN Operator*. [online] Available at: https://www.w3schools.com/sql/sql_between.asp [Accessed 17 Nov. 2025].